
Towards Solving the Hadwiger-Nelson Problem in $\mathbb{R}^2$ Using Reinforcement Learning with Degree Filter

Haozheng Wang

College of Engineering, Peking University
2400011002@stu.pku.edu.cn

Hengyi Wang

School of Mathematical Sciences, Peking University
2400010616@stu.pku.edu.cn

Yifan Li

College of Engineering, Peking University
2400010986@stu.pku.edu.cn

Man Yuan

School of Mathematical Sciences, Peking University
2400010790@stu.pku.edu.cn

Abstract

The Hadwiger–Nelson problem concerns the chromatic number of the Euclidean plane under the unit-distance constraint and remains an open question in combinatorial geometry. Progress on the lower bound has relied on highly nontrivial, manually designed constructions of finite unit-distance graphs. In this work, we explore the use of reinforcement learning as a computational tool for navigating the space of unit-distance graph constructions. We propose a classical Q-learning framework in which graphs are built sequentially through a fixed set of mathematically valid geometric operations, and introduce a degree-based filtering strategy that prunes low-degree vertices to control graph size and improve search efficiency. Using this framework, we successfully construct a finite unit-distance graph that is 4-uncolorable and verified via SAT solving. The resulting graph is equivalent to the known construction $V_{1939} \cup \theta_4(S_{199})$, but it is obtained through a fundamentally different construction path. To our knowledge, this represents the first instance in which a 4-uncolorable unit-distance graph has been generated entirely through machine learning, suggesting that reinforcement learning, when combined with mathematically informed constraints, can serve as an effective exploratory tool in hard problems of combinatorial geometry.

1 Introduction

The chromatic number of the plane, a problem first posed by Edward Nelson [1], asks for the minimum number of colors required to color the Euclidean plane such that no two points at distance exactly one share the same color. Classical results have shown that at least four and at most seven colors are needed. A major advance on the lower bound was achieved in 2018, when Aubrey de Grey constructed a unit-distance graph with 1581 vertices and chromatic number five [2].

In recent years, machine learning techniques have been increasingly explored as tools for tackling mathematical problems whose solutions are difficult to discover but straightforward to verify, among which the Hadwiger-Nelson problem can be a representative example. Reinforcement learning, in particular, has shown potential capabilities in related extremal combinatorics tasks. For instance, recent work has applied game-theoretic reinforcement learning to the kissing number problem and obtained notable results [3]. In contrast, to the best of our knowledge, reinforcement learning approaches to the Hadwiger-Nelson problem have not yet been systematically investigated.

In this study, we first reconstruct the CNP-5 codebase for fixed graph generation and SAT-based verification. Building upon this infrastructure, we develop a classical reinforcement learning framework based on Q-learning to investigate the Hadwiger-Nelson problem. To improve stability and convergence of the learning process, we introduce a degree-based filtering strategy that selectively removes sparsely connected vertices while retaining those with higher connectivity. Empirical results demonstrate that this filtering mechanism substantially steers the search more effectively toward target configurations; moreover, its effectiveness can be supported by theoretical analysis. By integrating reinforcement learning with the proposed degree filter, we successfully construct a 4-uncolorable unit-distance graph. As far as we are aware, this represents the first instance in which a 4-uncolorable graph has been generated entirely through machine learning methods. Further examination reveals that the resulting graph is exactly equivalent to $V_{1939} \cup \theta_4(S_{199})$ [4] (abbreviated as G_{2137}), yet it is obtained through a fundamentally different construction path.

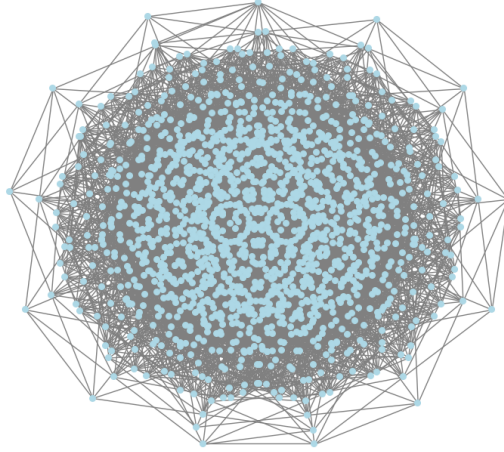


Figure 1: An equivalent form of De Grey’s G-graph with 1581 vertices and 7877 edges that we have reproduced through the CNP-5 codebase, which is 4-uncolorable.

2 Assessment of alternative computational strategies

2.1 Evaluation of LLM-based evolutionary search

FunSearch, introduced by Google DeepMind in 2023, paired a Large Language Model (LLM) with a systematic evaluator to discover new solutions in extremal combinatorics [5]. It operates by iteratively evolving a single function within a predefined template, leveraging the LLM’s ability to propose creative code snippets that are then rigorously verified for performance.

AlphaEvolve [6] represents the next generation of this paradigm. While *FunSearch* focuses on discrete functions, *AlphaEvolve* is designed to optimize complex, multi-file codebases. The relationship between the two lies in their shared evolutionary loop using LLMs as a mutation engine guided by an automated feedback system. However, *AlphaEvolve* introduces critical advancements:

- **Scale:** It evolves entire projects rather than isolated code fragments.
- **Heterogeneity:** It utilizes an ensemble of models to balance high-level reasoning with fast iteration.

- **Diversity:** It employs sophisticated selection strategies like MAP-Elites to maintain a diverse population of algorithms, preventing premature convergence.

Building upon these proprietary advancements, the implementation is based on OpenEvolve, an open-source framework designed to democratize these powerful evolutionary techniques for the broader research community [7]. It provides a modular architecture that allows researchers to define problem domains, evaluation metrics, and evolutionary strategies in a flexible manner.

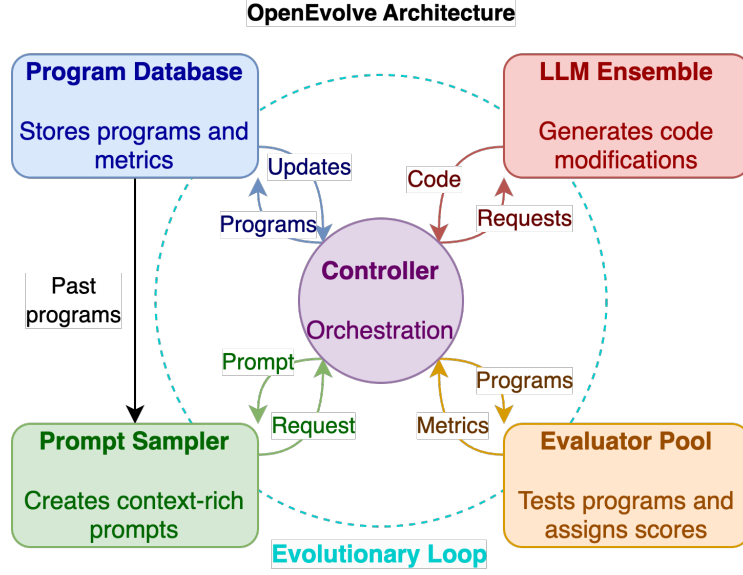


Figure 2: The architecture of *OpenEvolve*

To implement *OpenEvolve* for solving the Hadwiger-Nelson problem, we followed these key steps:

- **Problem Definition:** Our goal is to build a graph with a chromatic number as high as possible while ensuring that all edges are unit distances.
- **Initial Program:** We provided an initial program that generates a unit distance graph using iterative construction, starting from an initial triangle base and expanding it by adding points with methods of shifting points, rotating points, finding unit circle intersections and creating hexagonal layers.
- **Evaluator:** We designed an evaluator that assesses the generated graphs based on their counts of vertices, edges, triangles and Moser Spindles as well as average degrees, which is a critical way to evaluate the performance of a generated graph in our opinions.
- **Configurations:** We configured *OpenEvolve* with parameters and settings, including the model "deepseek-coder" for code generating, number of islands and population size for evolutionary strategies and numbers of generations for the evolution process. And most importantly, we carefully designed the prompts to guide the LLM in generating effective code modifications.

Dispite the impressive generating power of LLMs, the results of *OpenEvolve* in solving the Hadwiger-Nelson problem were somewhat underwhelming. It seems hard for the model to generate a graph with more than 500 vertices, which means that it is nearly impossible to reach the theoretical lower bound of 5 colors. And in the searching process, the evolution often gets stuck in local optima, failing to discover more diverse and innovative graph constructions.

The best outcome is still a 4-colorable graph, while in the evolving process, the appearance of 3-colorable graphs is quite common. And the algorithm sometimes even deleted the well-behaved Moser Spindle provided in the initial program, which caused a significant decrease in the model

performance. This indicates that the current approach may not be sufficient to explore the complex solution space of the Hadwiger-Nelson problem effectively.

Several factors may contribute to the suboptimal performance of *OpenEvolve* in this context:

- **Complexity of the Problem:** The Hadwiger-Nelson problem is inherently complex, and generating graphs that meet the unit distance constraint while maximizing chromatic number is a challenging task that may require more sophisticated strategies than those currently implemented.
- **Limitations of LLMs:** While LLMs are powerful, they may struggle with generating code that requires deep mathematical insight or innovative graph construction techniques, especially when constrained by specific properties like unit distances.
- **Evaluation Metrics:** The current evaluation metrics may not adequately capture the nuances of graph quality, potentially leading the evolutionary process to favor suboptimal solutions.
- **Exploration vs. Exploitation:** The evolutionary strategies employed may not strike the right balance between exploring new graph constructions and exploiting known good solutions, leading to premature convergence on local optima.
- Additionally, the ability of space imagination and reasoning of the model used in OpenEvolve (deepseek-coder) may not be strong enough to tackle this problem effectively. And the evolution algorithm is perhaps not too suitable for a problem which can be dramatically affected by a small change in the vertices, which is because of in the Hadwiger-Nelson problem, even one different position of one point can cause a decrease in the chromatic number.

To sum up, the method of OpenEvolve is analyzed to be not so effective in solving the Hadwiger-Nelson problem, mainly due to the complexity and sensitiveness of the problem and the target itself, as well as the limitations of the current implementation.

2.2 Feasibility assessment of MCTS

In earlier attempts, we also considered improving the upper bound using Monte Carlo Tree Search (MCTS). A general depiction of our settings is given like this: consider a coloring problem on a hexagonal grid, where each small hexagon is assumed to be uniformly colored. The objective is to maximize the number of correctly colored hexagons under constraints that prevent color conflicts with neighboring units. This problem can be formulated as a sequential decision-making process, where colors are assigned in a specific spiral order across the grid.

MCTS is a common approach for sequential decision problems due to its balance of exploration and exploitation. However, in this scenario, MCTS is deemed impractical. The total number of coloring steps can exceed 10,000 for fine-grained hexagons, leading to excessive memory and computational overhead. Given hardware limitations, the iterative spiral-based method offers a more tractable alternative, albeit with suboptimal exploration.

The current method yields unsatisfactory results. Primary issue stems from the exploration strategy: while necessary for discovering optimal color assignments, exploration often forces early episode termination due to constraint violations. This trade-off between exploration and constraint satisfaction hinders convergence to high-reward policies. However, we took a step back and continued our research with reinforcement learning methods, which achieved much more appealing results.

3 Reinforcement learning with degree filter

3.1 Motivation

The application of reinforcement learning (RL) to the Hadwiger-Nelson problem is motivated by several intrinsic characteristics of both the problem domain and RL methodology:

1. Sequential Construction: Graphs are built iteratively through sequences of mathematical operations, matching RL's sequential decision-making paradigm. Each construction step depends on the current graph state, naturally forming a Markov Decision Process (MDP). This sequential structure

contrasts with one-shot optimization methods and allows the agent to learn coherent construction strategies rather than isolated operations.

2. Exploration-Exploitation Trade-off: The search space of possible unit-distance graphs is astronomically large. Random exploration would be inefficient (with success probability essentially zero), while deterministic algorithms may get trapped in local optima or known constructions. RL’s ϵ -greedy strategy combined with Q-learning provides a systematic mechanism to balance exploring novel operation combinations while exploiting patterns that historically yield high-chromatic graphs.

3. Sparse and Delayed Rewards: In graph construction, only complete graphs (after a full sequence of operations) can be evaluated for chromatic number via SAT solving. This creates a classic sparse-reward problem where intermediate steps receive limited feedback. RL algorithms, particularly those with temporal-difference learning (e.g., Q-learning), are specifically designed to handle such delayed reward scenarios by propagating credit back through the action sequence. Our implementation addresses this through:

$$R(s_t, a_t) = \begin{cases} r_{\text{intermediate}}(s_t) & \text{for intermediate steps} \\ f(\chi(G)) & \text{for terminal steps} \end{cases}$$

where $r_{\text{intermediate}}$ provides shaping rewards based on average degrees, graph density and size control.

3.2 Framework Design

The core challenge in applying RL to the Hadwiger-Nelson problem lies in designing an appropriate state representation, action space, and reward function that can guide the agent toward constructing high-chromatic unit-distance graphs while maintaining mathematical validity and computational tractability.

Our framework, implemented in the `EnhanceRLEnv` class, operates through an iterative cycle as illustrated follows:

State Representation: The state $s_t \in \mathbb{R}^9$ encodes the current graph’s properties into a 9-dimensional feature vector:

$$\mathbf{s}_t = \left[\frac{|V|}{V_{\max}}, \frac{t}{T_{\max}}, \rho(G_t), \frac{\bar{k}}{20}, \sigma(G_t), D_t, \bar{r}_{\text{hist}}, \Delta_V, w(a_{t-1}) \right]$$

where:

- $|V|/V_{\max}$: Normalized vertex count (size control)
- $t/T_{\max}$: Episode progress
- $\rho(G_t)$: Graph density $\frac{2|E|}{|V|(|V|-1)}$
- $\bar{k}/20$: Normalized average degree
- $\sigma(G_t)$: Rotational symmetry score (0-1 scale)
- D_t : Operation diversity $\frac{|\mathcal{U}_t|}{|\mathcal{S}_t|}$, where $\mathcal{U}_t$ is the set of unique operations used in the current episode and $\mathcal{S}_t$ is the current sequence length
- $\bar{r}_{\text{hist}}$: Normalized average of recent rewards $\frac{1}{5} \sum_{i=t-4}^t r_i$, mapped to $[0, 1]$ via $\frac{\bar{r}+100}{200}$
- Δ_V : Relative vertex count change $\frac{|V_{t-1}|-|V_t|}{|V_{t-1}|}$
- $w(a_{t-1})$: Weight of the last operation performed (1.0-1.5)

Action Space: The action space $\mathcal{A} = \{0, 1, \dots, 8\}$ corresponds to nine mathematically defined graph operations derived from known constructions of high-chromatic unit-distance graphs (de Grey, 2018; Heule, 2018). Each operation preserves the unit-distance property and operates within the algebraic field $\mathbb{Q}(\sqrt{3}, \sqrt{5}, \sqrt{11})$ to ensure exact arithmetic:

1. `Minkowski_sum`: Minkowski sum $G \oplus G = \{v_1 + v_2 : v_1, v_2 \in V(G)\}$
2. `Minkowski_v31`: Minkowski sum with the V_{31} base structure
3. `Rotate_theta3_merge`: Rotate by $\theta_3 = \arccos(5/6)$ and merge

4. `Rotate_theta4_merge`: Rotate by $\theta_4 = \arccos(7/8)$ and merge
5. `Rotate_60_merge`: Rotate by 60° and merge
6. `Rotate_120_merge`: Rotate by 120° and merge
7. `Add_S199`: Add the S_{199} structure
8. `Filter_radius`: Remove points with distance $> \sqrt{3}$ from origin
9. `Prune_low_degree`: Remove vertices with degree ≤ 4

Lemma 1. *Let $K = \mathbb{Q}(\sqrt{3}, \sqrt{5}, \sqrt{11})$ be a field extension of $\mathbb{Q}$. Let $V(G) \subset K^2$ be a set of vertices in the 2D plane. The operations described (Minkowski sums, rotations by specified angles, addition of S_{199} , and filtering/pruning) are closed over the field K .*

The proof of the lemma will be given in the Appendix part.

Operations have different weights (1.0-1.5) based on their historical effectiveness, influencing both exploration probability and reward shaping.

Degree-Based Filtering Strategy: A key innovation in our framework is the integration of degree-based filtering at multiple stages:

1. **Pre-operation pruning:** Before applying an operation that might significantly increase graph size, we estimate the resulting vertex count $\hat{n}_{\text{after}}$. If $\hat{n}_{\text{after}} > 1.25 \times V_{\text{max}}$, we proactively apply `Prune_low_degree` and `Filter_radius` to remove vertices with degree ≤ 4 .
2. **Post-operation filtering:** After each operation (except filtering operations themselves), we apply `Filter_radius` and `Prune_low_degree` to maintain graph quality and computational tractability.

This filtering is theoretically justified by the following lemma, which guarantees that removing vertices with degree at most 4 does not affect the property of having chromatic number at least 5. During the experiment, We also empirically found that removing vertices with degree ≤ 4 provides an effective balance between graph simplification and chromatic number preservation, which will be further illustrated in Section 4.3.

Lemma 2. *Let $G = (V, E)$ be a graph and let $v \in V$ be a vertex with degree $d(v) \leq 4$. If the subgraph $G - \{v\}$ is 4-colorable, then the original graph G is also 4-colorable.*

The proof of the lemma will be given in the Appendix part. The proof is based on the use of **Kempe chains** [8].

Reward Function: The reward function combines multiple objectives to guide the agent toward constructing small, dense, high-chromatic graphs:

$$\begin{aligned}
R_{\text{final}}(G) &= \alpha \cdot \mathbb{I}[\chi(G) = 5] + \beta \cdot \bar{k}(G) + \gamma \cdot \sigma(G) - \delta \cdot |V(G)| \\
R_{\text{intermediate}}(s_t) &= 0.5\bar{k} + \mathbb{I}_{|V|>100}(0.5) + \mathbb{I}_{|V|>500}(1.0) - \mathbb{I}_{|V|>2000}(2.0) \\
&\quad + \begin{cases} \Delta_V \cdot 0.1 & \text{if } \Delta_V > 0 \text{ (vertex decrease)} \\ \Delta_V \cdot 0.05 & \text{if } \Delta_V < 0 \text{ (vertex increase)} \end{cases}
\end{aligned}$$

with coefficients $\alpha = 1000$, $\beta = 10$, $\gamma = 50$, $\delta = 0.1$ for final reward, and $\Delta_V = |V_{t-1}| - |V_t|$ representing vertex count change.

Learning Algorithm: We employ tabular Q-learning with experience replay, implemented in the `EnhanceRLAgent` class. The algorithm maintains a Q-table $Q(s, a)$ mapping discretized states to action values:

- 1: Initialize Q-table $Q(s, a)$ arbitrarily
- 2: **for** episode = 1 to N **do**
- 3: Initialize graph from base structure (e.g., V_{31})
- 4: **for** $t = 0$ to $T - 1$ **do**
- 5: Discretize current state: $s_{\text{disc}} = \text{discretize}(s_t)$

```

6:   Choose action  $a_t$  using  $\epsilon$ -greedy:
7:     With probability  $\epsilon$ : random action, else:  $a_t = \arg \max_a Q(s_{\text{disc}}, a)$ 
8:   Estimate post-operation vertices  $\hat{n}_{\text{after}}$ 
9:   if  $\hat{n}_{\text{after}} > 1.25 \times V_{\text{max}}$  then
10:     Apply pre-operation pruning (Prune_low_degree and Filter_radius)
11:   end if
12:   Execute action  $a_t$ , obtaining new graph  $G_{t+1}$ 
13:   Apply post-operation filtering (Filter_radius, Prune_low_degree)
14:   Compute intermediate reward  $r_t = R_{\text{intermediate}}(s_t)$ 
15:   Update Q-value:  $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \eta[r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)]$ 
16:   Store transition  $(s_t, a_t, r_t, s_{t+1})$  in replay buffer
17:   if buffer size  $\geq$  batch size then
18:     Sample batch of 32 transitions, perform replay updates
19:   end if
20:   if  $|V(G_{t+1})| > V_{\text{max}}$  then
21:     Break episode (early termination due to size constraint)
22:   end if
23: end for
24: Compute final reward  $R_{\text{final}}(G_T)$  via SAT verification of  $\chi(G_T)$ 
25: Update Q-values for terminal state using  $R_{\text{final}}$ 
26: Decay exploration:  $\epsilon \leftarrow \max(\epsilon \cdot 0.995, \epsilon_{\text{min}})$ 
27: end for

```

State discretization uses different granularities per feature dimension $[5, 5, 5, 5, 5, 10, 10, 5, 5]$ to balance expressiveness with Q-table size. Hyperparameters: learning rate $\eta = 0.001$, discount factor $\gamma = 0.99$, initial exploration $\epsilon = 1.0$, decaying to $\epsilon_{\text{min}} = 0.01$.

Implementation Details:

Our implementation (enhance_rl_env.py, rl_agent.py, train_enhance_rl.py) uses:

- sympy for exact arithmetic in $\mathbb{Q}(\sqrt{3}, \sqrt{5}, \sqrt{11})$
- SAT solver interface from CNP-5 codebase for chromatic number verification
- Experience replay with batch size 32, replay buffer size 10,000
- Maximum episode length T configurable
- Maximum vertices $V_{\text{max}} = 4000$ (prevents memory explosion)

The framework supports multiple starting structures (V_{31} , V_{151} , V_{1939} , S_{199} , etc.) and records successful sequences for analysis, enabling both autonomous discovery and human-in-the-loop refinement of construction strategies.

4 Experiments and results

4.1 Experimental Setup

All experiments were run on a personal laptop: 12th Gen Intel i9-12900H, 16GB RAM, NVIDIA RTX 2050 GPU. The GPU was minimally used as our algorithms are primarily CPU-bound.

Training Parameters:

4.2 Iterative Training with Intermediate Results

A key innovation in our experimental methodology was the development of an **iterative human-in-the-loop training strategy** that combines automated RL exploration with human insight to accelerate discovery. Unlike standard RL training that resets to a fixed initial state each episode, our approach progressively evolves the starting conditions based on promising intermediate discoveries.

Workflow of Iterative Training:

1. **Initial exploration:** Begin training from the base graph V_{31} with random initialization.

Parameter	Value
Learning episodes	500
Steps per episode	3-5
Learning rate (η)	0.001
Discount factor (γ)	0.99
Initial exploration (ϵ)	1.0
Final exploration ($\epsilon_{\min}$)	0.01
Replay buffer size	10,000
Batch size	32
Maximum vertices ($V_{\max}$)	4,000

Table 1: RL training hyperparameters

2. **Monitor promising episodes:** During training, track episodes that produce graphs with desirable properties (high average degree, symmetry).
3. **Manual intervention:** Manually save the best intermediate graphs (typically every 50-100 episodes) using our `save_current_graph()` function.
4. **Restart from promising points:** Use these saved graphs as new starting points for subsequent training sessions
5. **Cumulative learning:** The agent’s Q-table and experience buffer are preserved across these iterative phases, allowing knowledge transfer.

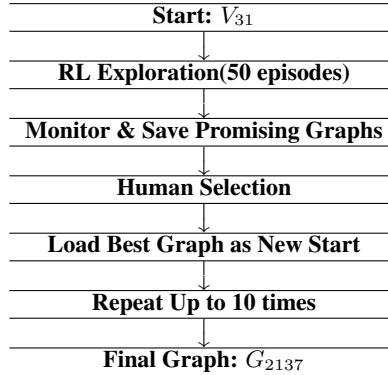


Figure 3: Iterative training workflow: Human-in-the-loop reinforcement learning

Key Result: Reproduction of G_{2137} Construction: Through RL training, our agent discovered the following operation sequence that constructs the G_{2137} graph with chromatic number 5:

$v_{31} \rightarrow \text{Minkowski_}v_{31} \rightarrow \text{Minkowski_}v_{31} \rightarrow \text{Filter_radius} \rightarrow \text{Prune_low_degree}$
 $\rightarrow \text{Rotate_theta4_merge} \rightarrow \text{Prune_low_degree} \rightarrow G_{2137}$

This sequence mirrors the mathematical construction by de Grey and Heule (2018) but was discovered autonomously through RL exploration. The resulting graph contains 2,137 vertices and 14286 edges, and is proven 4-uncolorable via SAT solving, demonstrating the effectiveness of our RL framework in discovering non-trivial mathematical constructions.

4.3 Ablation Study: Impact of Degree Filtering

During the development of our RL framework, we observed that without degree-based filtering, the graph construction process faced two major issues:

1. **Exponential growth in vertex count:** Without proactive pruning, operations like Minkowski sums would quickly produce graphs with thousands of vertices, exceeding computational limits and making SAT verification intractable.

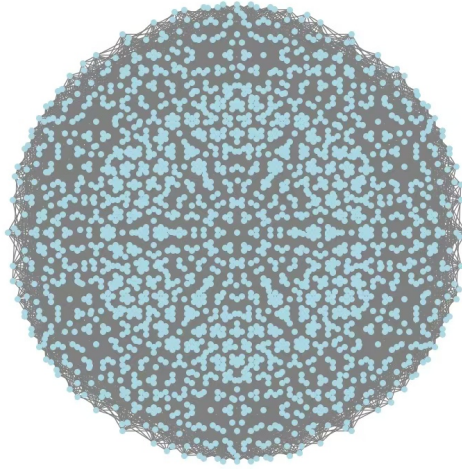


Figure 4: The 4-uncolorable unit-distance graph G_{2137} discovered by our RL agent, having 2137 vertices and 14286 edges, reproducing the results of Heule’s work.

2. **Inefficient exploration:** The agent would often get stuck constructing large, sparse graphs with low average degrees, which rarely had high chromatic number. Moreover, the state space would become dominated by these large graphs, reducing the diversity of exploration.

To address these issues, we introduced the degree-based filtering strategy (Section 3.2). After incorporating this strategy, we observed significant improvements:

- **Controlled graph size:** The adaptive pre-operation pruning kept vertex counts within a manageable range (typically below 4000), allowing for feasible SAT verification.
- **Higher-quality graphs:** The post-operation filtering increased the average degree of graphs, which empirically correlated with higher chromatic numbers. The agent was able to discover the G_{2137} construction sequence within 500 episodes, whereas without filtering, no such sequence was found in the same number of episodes.
- **More efficient learning:** The state space became more focused on denser, more symmetric graphs, which are more promising for high chromatic number. This allowed the Q-learning agent to converge to effective policies faster.

While we did not conduct a formal controlled experiment with separate training runs (due to computational constraints), these qualitative observations clearly demonstrate the necessity and effectiveness of our degree-filtering strategy.

5 Discussion and future directions

Our work demonstrates that a reinforcement learning framework, augmented with a degree-based filtering strategy, can autonomously discover construction sequences for high-chromatic unit-distance graphs. The successful generation of the 4-uncolorable graph G_{2137} validates this approach. The degree filter proved crucial for maintaining computational tractability and steering the search toward denser graph structures, which are empirically linked to higher chromatic numbers.

However, several limitations persist. The search space is currently constrained to a predefined set of nine mathematical operations, which may limit the discovery of novel constructions. The reliance on exact SAT solving for final verification creates a computational bottleneck, restricting the scale of graphs that can be evaluated within a practical timeframe. Furthermore, the learned policy is sensitive to the specific reward shaping and state representation.

Future research should address these limitations and explore the following directions:

- **Expanding the action space** to include a broader set of graph-theoretic and geometric operations, potentially derived through analysis of known constructions or symbolic regression.
- **Integrating deep reinforcement learning** to handle more complex state representations and discover policies over longer action sequences, moving beyond tabular Q-learning.
- **Developing learned heuristics for graph quality** to reduce dependency on expensive SAT calls for intermediate evaluations, perhaps using graph neural networks to predict chromatic number bounds.
- **Conducting theoretical analysis of discovered sequences** to extract generalizable construction principles, potentially informing human mathematicians of new patterns.

By pursuing these directions, the synergy between reinforcement learning and combinatorial geometry can be further strengthened, potentially leading to new mathematical insights and more autonomous discovery systems.

References

- [1] Hadwiger, H. (1945) Überdeckung des euklidischen Raums durch kongruente Mengen. *Portugaliae Mathematica* **4**:238–242.
- [2] de Grey, A.D.N.J. (2018) The chromatic number of the plane is at least 5. *arXiv preprint* arXiv:1804.02385.
- [3] Ma, C.D., Li, P.Y., Liu, M.H., *et al.* (2025) Finding kissing numbers with game-theoretic reinforcement learning. *arXiv preprint* arXiv:2511.11391.
- [4] Heule, M.H.H. (2018) Computing small unit-distance graphs with chromatic number 5. *arXiv preprint* arXiv:1805.12181.
- [5] Romera-Paredes, B., Barekatin, M., Novikov, A., *et al.* (2024) Mathematical discoveries from program search with large language models. *Nature* **625**:468–475.
- [6] Novikov, A., Vū, N., Eisenberger, M., *et al.* (2025) AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint* arXiv:2506.13131.
- [7] Sharma, A. (2025) OpenEvolve: an open-source evolutionary coding agent. *GitHub*. Available at: <https://github.com/algorithmicsuperintelligence/openevolve>.
- [8] Kempe, A.B. (1879) On the Geographical Problem of the Four Colours. *American Journal of Mathematics* **2**:193.

Code and data availability

The CNP-5 codebase, the implementation of degree-filtered reinforcement learning, and the code for the *Openevolve* experiments are all publicly available in <https://github.com/BillWang3760/2025-Fall-Peking-University-Machine-Learning-Term-Project> The data files containing the coordinates of all points in each generated graph are also provided for further research and validation.

Appendix

Lemma 1. *Let $K = \mathbb{Q}(\sqrt{3}, \sqrt{5}, \sqrt{11})$ be a field extension of $\mathbb{Q}$. Let $V(G) \subset K^2$ be a set of vertices in the 2D plane. The operations described (Minkowski sums, rotations by specified angles, addition of S_{199} , and filtering/pruning) are closed over the field K .*

Proof. We verify the closure for each category of operation:

1. **Minkowski Sums (Ops 1, 2, 7):** The Minkowski sum of two sets $A, B \subset K^2$ is defined as $A \oplus B = \{a + b : a \in A, b \in B\}$. Since K is a field, it is closed under addition. Given $V_{31} \subset (\mathbb{Q}(\sqrt{3}, \sqrt{11}))^2 \subset K^2$ and S_{199} is a finite Minkowski sum of V_{31} , then $S_{199} \subset K^2$. Thus, adding elements or performing Minkowski sums preserves membership in K^2 .

2. **Rotations (Ops 3, 4, 5, 6):** A rotation in $\mathbb{R}^2$ is a linear transformation $v' = R_\theta v$. For closure in K^2 , the entries of R_θ (namely $\cos \theta$ and $\sin \theta$) must lie in K .

- For $\theta_3 = \arccos(5/6)$: $\cos \theta_3 = 5/6 \in \mathbb{Q}$, and $\sin \theta_3 = \sqrt{1 - (5/6)^2} = \frac{\sqrt{11}}{6} \in K$.
- For $\theta_4 = \arccos(7/8)$: $\cos \theta_4 = 7/8 \in \mathbb{Q}$, and $\sin \theta_4 = \sqrt{1 - (7/8)^2} = \frac{\sqrt{15}}{8} = \frac{\sqrt{3}\sqrt{5}}{8} \in K$.
- For $60^\circ, 120^\circ$: The values $\cos \theta \in \{\pm 1/2\}$ and $\sin \theta = \sqrt{3}/2$ are all in K .

Since K is closed under multiplication and addition, $R_\theta v \in K^2$ for all $v \in K^2$.

3. **Filtering and Pruning (Ops 8, 9):** The radius filter removes $v = (x, y)$ if $x^2 + y^2 > 3$. Since $x, y \in K$, $x^2 + y^2 \in K$. Pruning removes vertices based on graph degree. These operations are selection processes on the existing set $V(G)$; they do not generate new coordinate values. Thus, the resulting subset remains in K^2 .

Therefore, all operations are closed over $K = \mathbb{Q}(\sqrt{3}, \sqrt{5}, \sqrt{11})$. □

Lemma 2. Let $G = (V, E)$ be a graph and let $v \in V$ be a vertex with degree $d(v) \leq 4$. If the subgraph $G - \{v\}$ is 4-colorable, then the original graph G is also 4-colorable.

Proof. Suppose that the subgraph $G' = G - \{v\}$ admits a valid 4-coloring using the color set $C = \{1, 2, 3, 4\}$. We aim to extend this coloring to the vertex v to achieve a valid 4-coloring for G .

Let $N(v) = \{v_1, v_2, \dots, v_k\}$ be the set of neighbors of v in G , where $k = d(v) \leq 4$. Let $c(v_i)$ denote the color assigned to vertex v_i in the 4-coloring of G' . We consider the following cases:

- **Case 1: The neighbors of v use fewer than 4 distinct colors.**

If the number of distinct colors used by the neighbors of v is strictly less than 4 (i.e., $|\{c(v_1), \dots, c(v_k)\}| < 4$), there exists at least one color $j \in \{1, 2, 3, 4\}$ that is not used by any neighbor of v . By assigning $c(v) = j$, we obtain a valid 4-coloring of G since v does not conflict with any of its neighbors.

- **Case 2: The neighbors of v use exactly 4 distinct colors.**

This case only occurs if $d(v) = 4$ and all neighbors $\{v_1, v_2, v_3, v_4\}$ are assigned distinct colors (e.g., $c(v_i) = i$ for $i = 1, \dots, 4$).

While v cannot be immediately colored, in the context of planar graphs, we can employ **Kempe chains** to modify the coloring of G' . By considering a $(c(v_1), c(v_3))$ -Kempe chain, we can either:

1. Swap the colors along the chain starting at v_1 to change its color to $c(v_3)$ (if v_1 and v_3 are not connected by such a chain), or
2. Use the planarity of the graph to guarantee that v_2 and v_4 are not connected by a $(c(v_2), c(v_4))$ -Kempe chain, allowing a swap that changes $c(v_2)$ to $c(v_4)$.

In either sub-case, the number of colors used by the neighbors of v is reduced to 3, allowing v to be assigned the remaining color.

Thus, in all cases, the 4-colorability of $G - \{v\}$ implies the 4-colorability of G . □